

XML-izing Mizar: Making Semantic Processing and Presentation of MML Easy

Josef Urban

Dept. of Theoretical Computer Science, Charles University,
Malostranské nám. 2/25, Praha, Czech Republic
`urban@kti.ms.mff.cuni.cz`

Abstract. Since version 7.2 the Mizar system produces quite detailed XML-based semantic description of Mizar articles during their verification. This format is now used natively for most of the processing done by Mizar, e.g., also for the whole Mizar internal database. The main motivation for switching to this XML-based representation is to make semantic communication with Mizar and presentation of the MML more accessible to a variety of external tools and systems. This article briefly describes this format and its current implementation, and shows examples of its usage. These examples include presentation of linked Mizar articles in modern XML-capable browsers like Mozilla, authoring assistance in the Mizar mode for Emacs, and experiments with XML-based querying languages like XQuery over the Mizar Mathematical Library loaded into a native XML databases like eXist. This work makes the currently largest repository of formal mathematics available to many kinds of presentational, data-mining, and automated reasoning applications and experiments, and the goal of this article is also to encourage, facilitate and provide recipes for the development of such applications.

1 Motivation

The Mizar¹ [Rud92, RT99] system is today probably the longest living proof checker of the world. There have been several re-implementations of this system during the previous decades [MR04], the current version is Mizar 7. These re-implementations were usually triggered by changes in the Mizar language or in the core proof-checking algorithms, coming from Mizar-based research on formalization of mathematics. Unlike these changes, the XML-ization of Mizar described here was triggered and also largely made possible by external influences. In this section we state the reasons that motivated this work.

1.1 Standard Format for External Semantic Processing of MML

First, a number of Mizar-external tools working with the semantic content of the Mizar Mathematical Library (MML) have been implemented in the recent years: MML Query [BR03], MoMM [Urb05b], MPTP and Mizar Proof Advisor [Urb04],

¹ <http://mizar.uwb.edu.pl>

semantic browsing in Emacs [BU04, Urb05a], etc. Such projects have shown the general interestingness and usefulness of Mizar-external semantic processing of the MML. However, since the Mizar language attempts to be very close to mathematical vernacular, the raw Mizar articles are difficult to process without a good knowledge of the Mizar parser. That is why all the above mentioned systems are not completely Mizar-external. They usually have to include some kind of a *Mizar exporter*, i.e. a tool written on top of the Mizar parser that exports various parts of the Mizar texts into the format used by these systems. Having to write such a tool can however be a “showstopper” for a number of people interested in the semantic processing of MML. It already requires some effort to get to know the Mizar language, and it requires even more effort to gain some working knowledge of the Mizar implementation. The obvious remedy is to export the semantic content of MML into an easily parsable and well-described format. The first effort that largely succeeded in this was the ILF project [Dah98], for which Czeslaw Bylinski wrote a tool exporting Mizar articles into a well-described Prolog format. Today, XML seems more suitable for such an export, because of its large-scale adoption throughout the software industry, resulting in the availability of a number of standards and tools for its description, validation, parsing, transforming, presentation, storage and indexing, etc.

1.2 Better Format for Internal Mizar Processing

The second reason for XML-izing Mizar comes from the Mizar implementation itself. The fast progress in the computer industry has exponentially increased the amounts of memory and storage space available to current systems, when compared with the situation thirty years ago. It is no longer necessary to devise as succinct formats as possible for storing internal information. More attention can be paid to the maintainability, extendability, standardization, and ease of use of such formats. Modern computer languages have gone as far as having default procedures for storage/loading complex data in XML.

The old Mizar format for internal data storage suffered from the deficiencies mentioned above. It was so succinct that it was quite hard to extend, and therefore changes to the Mizar language usually meant painful low-level work on data-storage. Only as little information as necessary for the basic verification was often stored. It meant that utilities needing more information were hard to implement, or entirely impossible without a large-scale duplication of code. While the verbosity has to be controlled even today, we can afford to have much more complete information at any time of the Mizar processing with no significant impact on its speed, and thus allow an easy implementation of a number of new Mizar utilities.

1.3 Putting It Together

The distinction between external and internal reasons for this work is not perfect. It has become quite clear during the years of Mizar development that various special-purpose Mizar-exporting tools written for external processing of MML

usually become very quickly outdated. The only working way how to avoid it is to maintain one common and well-described format both for internal Mizar processing and for the external tools. Additionally, the distinction between internal and Mizar-external tools is not perfect either. Some tools (e.g., the MizarMode [Urb05a]) eventually became a standard part of the Mizar distribution, while some other Mizar utilities are no longer distributed. It is quite realistic to predict that in some time e.g. more searching tools might be distributed with Mizar, especially now that the internal Mizar database is already queryable using XML standards like XPath and XQuery.

2 Description of the Mizar XML Implementation

The code-base of the Mizar kernel is quite large (ca. 50000 lines). Since Mizar is a “production” system used by many people around the world, the switch had to be done in a conservative way, introducing as little user-visible changes to Mizar as possible. The work has been done in three smaller phases, making the transition smoother and better controlled. After each phase, regression testing on both the MML and randomly created incorrect articles was carried out before releasing the new Mizar version. The first two phases done in 2004 dealt with the old information passing in Mizar. As noted above, these old formats were devised in times when memory and storage space were expensive, and therefore a lot of useful information was not present in them. Additionally, different I/O procedures were used for the same data-structures in different contexts, again in order to save memory and storage. In the third phase the actual switch to XML format was carried out.

2.1 Providing Semantic Form of Mizar Articles

In the first phase the interface between semantic analysis and proof checking was completely rewritten. The output of semantic analysis is now a complete semantically disambiguated form of a Mizar article, while earlier it included only the information that was absolutely necessary for proof checking. The additional information includes, e.g., the description of the Jaskowski-style natural deduction steps used in the proofs, the modified thesis after each deduction step, definitional macros (definientia) used for silent unfolding of thesis, etc. The semantic analysis and proof checking became completely separated, and they can now even be run as separate programs. Though this is not very important for Mizar at the moment, it documents well the conceptual separation that was carried out: on one hand we have the parser for the natural-language-like Mizar articles, and on the other hand we have the Mizar verifier and other utilities working on the easily machine-processable format produced by the parser. It is interesting to observe that such a split is becoming more and more common in many “authoring” areas, and various human-friendly compact representations (e.g. wiki for HTML, compact syntax for RELAX NG, etc.) are increasingly used to produce richly annotated machine-processable data. In this aspect, the Mizar language might be thought of as a similar compact syntax for human-authored formal

mathematics, and the detailed output produced by the Mizar parser (including semantic analysis) as its machine-processable equivalent.

This first step has already allowed to rewrite more cleanly a number of Mizar utilities, and to produce some interesting new utilities. It is now possible to collect statistics about the usage of definitional macros in MML, or to easily compute all the suppositions done along the Jaskowski-style proof path to some lemma. The latter can be used e.g. in the MoMM data-miner [Urb05b] as another method of exporting internal lemmas from their proof context. As noted in the Motivation section, a big incentive for this step was also the need for processing of complete Mizar proofs in external systems like MPTP, MML Query, Mizar Proof Advisor and others. However once the first step was finished, it became clear that the complete XML-ization is really feasible in quite a short time, and work on these external systems was largely postponed after that.

2.2 XML-izing Mizar

Mizar can be used on a number of computer architectures, and can be compiled with all major objective Pascal compilers (FPC, GPC, Delphi, Kylix). This variety makes it quite difficult to use some external XML-parsing library for Mizar. So the starting point that triggered further work on XML in Mizar was the implementation of a native “Tiny XML” Pascal parser by Czeslaw Bylinski. This fast parser now implements just a subset of XML needed by Mizar, more advanced features like parsing of XML entities are not yet supported (and probably not necessary for our purpose at all). One thing that still prevented the switch to XML was the number of memory-optimized versions of essentially the same data-structure in Mizar. During the second phase a large unification of such Mizar data-structures was done and the Mizar code-base thus simplified quite a lot (about 3000 lines were pruned). The effects of both the first and this second phase on the speed, memory, and storage requirements of Mizar processing were (according to current standards) really negligible. For instance, the slowdown caused by having the detailed information after semantic analysis was about 1-2 percent. Most of the Mizar processing time is today used for the proof checking, and various modifications of the proof checking module usually cause much higher deviations (in both directions) than this number.

In the third phase, an XML representation was suggested for most of the Mizar data-structures, and most of the I/O procedures have been rewritten for this representation. The initial task of the XML format² was just to allow the switch to XML, and it will probably go through some more changes in the future. The applications that we describe below are already giving quite a lot of feedback about the usability of this format for Mizar-external purposes, and some of that feedback already resulted in changes in the XML format. The general policy concerning such changes is to incrementally improve the clarity of the

² Since Mizar version 7.4, the Mizar XML documentation is distributed with Mizar. It can be also viewed at <http://lipa.ms.mff.cuni.cz/~urban/Mizar.html>

XML representation, so that it is more useful for various external applications, however this has to be done without serious influence on the Mizar processing. A lot of information can now be added to the XML representation by external processing via simple XSLT stylesheets (we give an example below). If a really rich format is required by some external tool, the developers of such a tool should use XSLT to produce such a rich format from the basic Mizar XML. On the other hand, most of the Mizar XML texts is now occupied by Mizar terms, formulas and types, so adding markup to other Mizar constructs is usually quite cheap and with negligible influence on the speed, memory and storage requirements of the Mizar processing.

The description of the Mizar XML format is (following Knuth ideas on literate programming) now given directly in the Mizar code, as specially tagged Pascal comments. They are written in the RELAX NG compact syntax³ which is (unlike other XML-based specification formats⁴) natural, human-friendly, and actually useful even as an internal description of the Pascal data-structures. It would not be feasible to fully produce the I/O routines automatically from the RELAX NG descriptions (or vice-versa), however, at least a part of the Mizar XML parsing code is now produced automatically from them. This, together with validation by the resulting schemata, should reasonably guarantee that the descriptions will be maintained when Mizar changes, and will always faithfully represent the Mizar XML format. These descriptions are then very simply (by a Perl one-liner) collected into the file `Mizar.rnc`, which is now distributed with Mizar. The particular RELAX NG schemata for each XML file produced by Mizar are then created just by stating the starting element, and including `Mizar.rnc`. For instance, the file `article.rnc` describing the complete semantic content of Mizar articles looks this way:

```
include "Mizar.rnc"
start = Article
```

2.3 Speed and Space Considerations

The switch to XML had its effects on the speed and storage requirements of Mizar processing. The following table 1 compares the speed of verification and accommodation⁵ on complete MML and on average article before and after XML-ization. The verification time of MML increased from ca. 47 minutes to ca. 78 minutes, i.e., about 1.67 times. This is the most important number for Mizar users, because verification is carried out very frequently during authoring Mizar articles. The accommodation time grew quite considerably, from ca. 2 minutes to about 15 minutes. Accommodation is however quite rare operation,

³ <http://www.oasis-open.org/committees/relax-ng/compact-20021121.html>

⁴ “XML combines all the inefficiency of text-based formats with most of the unreadability of binary formats.” – Oren Tirosh, `comp.lang.python`.

⁵ Accommodation extracts from the internal Mizar database the Mizar items used for processing a new article, and creates a local environment for that article.

Table 1. Comparison of Mizar processing times (in seconds) before and after XML

Action	MML bef. XML	MML after XML	avrg. bef. XML	avrg. after XML
verification	2832	4723	3.2	5.34
accommodation	145	906	0.16	1.02

Table 2. Comparison of the sizes of exported Mizar files (in Kb) before and after XML

MML bef. XML	MML after XML	avrg. bef. XML	avrg. after XML
12270	78080	14	89

and in absolute numbers it is still much below the verification times. Since accommodation spends most of its time in I/O procedures, its time increase corresponds quite faithfully to the growth in the size of processed data, i.e. to the size of the internal database. These sizes are compared in table 2. This increase is considerable (636 percent), and it is obviously caused by the greater verbosity of the XML representation. It would be possible to compress the XML format quite a lot (and therefore also decrease the processing times) by choosing much shorter names of elements and attributes, but it would make the XML format less understandable. Today (spring 2005), one gigabyte of storage costs about 0.75 USD, so from this point of view, an increase by tens of megabytes is really negligible, and should pose no problems to users. The most sensitive value is therefore the verification time, and this is the reason why the verbosity of the XML format needs to be controlled. On the other hand, the Moore's law will take care of the current verification time increase in one year, and as mentioned above, the deviations caused by modifications of the proof checking module are quite frequent and at about the same scale.

2.4 Short Description of the Mizar XML Format

Long before XML and XML-based mathematical formats were devised, Mizar had strict internal division between the presentation layer and the semantic layer. The current Mizar XML format captures only the semantic content of Mizar articles, which is used for Mizar verification and also in practically all the above mentioned Mizar-external tools. As in e.g. MathML, it is possible to construct the presentation from the Mizar semantic layer, and e.g. MML Query has been doing this for several years. However, such reconstructed presentation often differs from the original presentation used by authors of the articles. So the current XML format cannot be used yet for exact presentation purposes, like the Journal of Formalized Mathematics⁶. Therefore the Mizar presentation layer will probably also be XML-ized in the future, it has been already discussed among Mizar developers several times.

⁶ <http://mizar.uwb.edu.pl/JFM/>

As noted above, most of the markup is now occupied by Mizar terms, formulas and types. Terms and formulas are now just named patterns representing the various elements:

```
Term =
  ( Var | LocusVar | FreeVar | LambdaVar | Const | InfConst | Num
    | Func | PrivFunc | Fraenkel | QuaTrm | It | ErrorTrm)
```

```
Formula =
  ( Not | And | For | Pred | PrivPred | Is | Verum | ErrorFrm )
```

These particular term and formula elements are described in more detail in the RELAX NG documentation⁷. Note that the logical connectives used in the Mizar semantic layer are only **Not**, **And** and **For**, all other connectives are transformed to them. The markup for Mizar types looks this way:

```
## Adjective is a possibly negated (and parameterized) attribute
Adjective = element Adjective {
  attribute nr { xsd:integer },
  attribute value { xsd:boolean }?,
  Term*}

## Cluster of adjectives
Cluster = element Cluster {Adjective*}

## Parameterized type - either mode or structure
## First goes the LowerCluster, then UpperCluster
Typ = element Typ {
  attribute kind { "M" | "G" | "errortyp" },
  attribute nr { xsd:integer }?,
  Cluster*, Term* }
```

The building blocks of Mizar proofs are Jaskowski-style [Jas34, Pel99] natural deduction steps⁸. Those modifying the currently proved *thesis* are **SkeletonItems** and the rest are called **AuxiliaryItems**. Again, these are just named patterns, the particular elements are described in the documentation:

```
SkeletonItem =
  ( ( Let | Conclusion | Assume | Given | Take | TakeAsVar ), Thesis? )

AuxiliaryItem =
  (JustifiedProposition | Consider | Set |Reconsider | DefFunc | DefPred)
```

The top-level **Article** element describes the semantic content of a whole Mizar article. It contains various Mizar blocks, introducing definitions, theorems, schemes, etc.:

⁷ <http://lipa.ms.mff.cuni.cz/~urban/Mizar.html>

⁸ Actually, a new name like *Mizar-Jaskowski* or *Jaskowski-Trybulec ND style* might be more proper, since the Jaskowski style was reinvented and augmented in Mizar.

```
## The complete article after analyzer, aid specifies its name in uppercase.
Article = element Article {
  attribute aid { xsd:string },
  ( DefinitionBlock | RegistrationBlock |
    NotationBlock | Reservation | SchemeBlock |
    JustifiedTheorem | DefTheorem | Definiens |
    Canceled | Pattern | AuxiliaryItem )* }
```

This semantic description of an article is now automatically constructed during the verification, so the way to produce such description for the whole MML from installed Mizar distribution is e.g. following⁹:

```
mkdir tmp; cp -r $MIZFILES/mml tmp; cd tmp/mml
for i in `cat $MIZFILES/mml.lar`; do accom $i; verifier $i; done
```

The Mizar distribution also includes the Mizar internal database (directory `$MIZFILES/pre1`), which is now also fully XML-ized. This database contains all the MML items that can be re-used in other articles, i.e., theorems, definientia, schemes, notations, registrations and constructors¹⁰. Again, all these files can be validated by setting a suitable starting element (i.e., `Theorems`, `Definientia`, `Schemes`, `Notations`, `Registrations` and `Constructors`) and including the basic schema `Mizar.rnc`. Despite the large-scale code unification described in 2.2, the markup used for higher-level structures in the internal database is not yet completely the same as the markup used for the description of whole articles. E.g., the `Scheme` element used in the internal database differs quite considerably from the scheme description used in whole articles inside `SchemeBlocks`. Complete reconciliation of all such differences would be too time-consuming, and the plan is to reconcile them gradually in the future, also using the feedback obtained from various applications building on the XML format.

3 Applications

3.1 Stylesheet Processing

XSL Transformations (XSLT¹¹) is a high-level declarative language used today very frequently for transforming XML documents. A number of fast XSLT processors are now publicly available, e.g., `xsltproc`¹² or `xalan`¹³. XSLT is often used for producing customized HTML from XML documents, and state-of-the-art browsers like Mozilla and Firefox are shipped with built-in XSLT processors. Useful compact syntaxes exist for human-friendly authoring of XSLT stylesheets, the XSLTXT¹⁴ compact syntax is used to produce the stylesheets presented here.

⁹ This will take ca. 1-2 hours on standard hardware.

¹⁰ Just for completeness, the XML-ized form of the file `$MIZFILES/mml.vct` containing MML symbols will be probably added to Mizar distribution.

¹¹ <http://www.w3.org/TR/xslt>

¹² <http://xmlsoft.org/XSLT/xsltproc2.html>

¹³ <http://xml.apache.org/xalan-c/>

¹⁴ <http://www.zanthan.com/ajm/xsltxt/>

Some of these stylesheets are now distributed with Mizar, others are available from the author's site¹⁵.

A typical task easily achieved by XSLT is to produce a richer XML description of an article from the description supplied by the Mizar parser. During accommodation, Mizar collects from the internal database various resources required for a new Mizar article. Such resources (theorems, schemes, constructors, etc.) are organized into arrays for fast lookup, and the current XML format only uses their positions in these arrays (one integer) instead of their absolute MML address. Using the absolute MML addresses makes the resulting XML files almost twice as big, and as described above, this has negative effect on the speed of Mizar verification and accommodation. The shortened and the absolute description of the basic Mizar type **set** (with no adjectives) are shown below:

```
<Typ kind="M" nr="1">
  <Cluster/>
</Typ>
```

```
<Typ kind="M" nr="1" aid="HIDDEN" absnr="1">
  <Cluster/>
</Typ>
```

The additional attributes **aid** and **absnr** encode (together with the attribute **kind**) the absolute MML address of this resource, i.e. it is the first mode type introduced in article **HIDDEN**.

On the other hand, having the absolute addresses directly inside the XML is often very useful, and it is indispensable e.g. when one wants to uniformly process multiple articles and their resources. For instance, provided that we use the absolute addresses, the following XPath expression will return all adjectives defined in article **ORDINAL1** used in generalizations (**Let**-items) in articles **CARD_1** and **CARD_2**.

```
/Article[(@aid="CARD_1")or(@aid="CARD_2")]//Let//Adjective[@aid="ORDINAL1"]
```

Addition of the absolute addresses e.g. to Mizar adjectives can be done using the following XSLT piece of code:

```
tpl [Adjective] { <Adjective
  { copy-of '@*'; abs(#k="V", #nr='@nr'); apply; }}
```

The XSLT function **abs(#k, #nr)** just looks up the absolute address (in the file containing the lookup table) using the serial number as a key. The complete transformation adding absolute addresses to all Mizar resources takes about 100 lines of XSLT¹⁶.

3.2 Browsing Semantic Form of Articles in XSLT-Capable Browsers

One of the original design goals of XML was to “become a better HTML”. Several popular internet browsers are able to apply XSLT stylesheets to a given XML file

¹⁵ <http://kti.ms.mff.cuni.cz/cgi-bin/viewcvs.cgi/xsl4mizar/>

¹⁶ <http://kti.ms.mff.cuni.cz/cgi-bin/viewcvs.cgi/xsl4mizar/addabsrefs.xslt.txt>

and display the result as if it was a normal HTML document. Since the XML is now produced after each verification run, direct browsing of the semantic form of the currently authored Mizar article is just a matter of specifying a suitable stylesheet. If such a stylesheet is present, no other XML tools need to be installed, the only necessary step is to open the XML file in a browser like Mozilla or Firefox after the verification.

An initial stylesheet `miz.xmltxt` serving this purpose has been developed¹⁷, and since Mizar version 7.4 it is also distributed with Mizar. The default installation location of this stylesheet is written to the XML output of the Mizar parser, which means that the XML output can be displayed locally, without internet connection. The default linking of Mizar resources is however done to the internet address of MML Query, so for looking up these resources in the default way an internet connection is necessary. There are several other experimental linking possibilities, including linking to “self”, i.e. to the collection of XML files produced by the verification of MML. Another good linking possibility would arise if we (the Mizar developers) decided to distribute a richer version of the internal Mizar database, containing the absolute MML addresses as described above. This version is about twice as big (ca. 100 MB), which would cause further slowdown of accommodation, but the Mizar distribution would then also directly serve as a self-sufficient HTML documentation of the semantic form of MML abstracts, similar to the MML Query generated abstracts [BU04].

All the HTML files produced by applying the default `miz.xmltxt` stylesheet to the XML description of MML articles can be now browsed at the author’s site¹⁸. Adding number of presentational features like e.g. JavaScript hiding/showing of subproofs to `miz.xmltxt` should be very easy, and users and developers are encouraged to use `miz.xmltxt` as a building block for such richer presentations.

3.3 Disambiguating in MizarMode

External tools are not the only uses of the semantic representation of Mizar articles. In more advanced mathematical domains, notation from several other domains are combined, sometimes causing ambiguities. It is often nontrivial to guess how certain terms and formulas are parsed by Mizar, especially in more complex Mizar articles.

In [BU04, Urb05a] we describe a solution (called *Constructor Explanations*) to this problem in the Emacs authoring environment for Mizar (MizarMode), using the MML Query generated abstracts and the old-style Mizar parser output. As noted above, the old output contained only as little information as was absolutely necessary for proof checking, and also complete parsing of its custom format would require detailed rewrite of the Pascal I/O procedures into Emacs Lisp. Both these reasons had lead to a middle-way approach: only Mizar propositions with *SimpleJustification* (XML element `By`) were parsed by Emacs from the Mizar parser output and made available for semantic disambiguation.

¹⁷ <http://kti.ms.mff.cuni.cz/cgi-bin/viewcvs.cgi/xml4mizar/miz.xmltxt>

¹⁸ <http://lipa.ms.mff.cuni.cz/~urban/xmlmml/>

Both these reasons have now disappeared: a complete semantic representation of an article is now available, and it is in XML. Emacs is distributed with a default XML-parsing module since version 21.1. The *Constructor Explanations* are now therefore applicable virtually to any Mizar construct. In practice, the *Constructor Explanations* however use the mechanism of Emacs text properties, to couple pieces of text with their disambiguated form. This coupling is easily done for all XML constructs that remember their original position in the article, so it now covers all Mizar **Propositions**. Many other XML constructs remember their position too, so *Constructor Explanations* can be easily extended to them. The only additional work needed for this is to specify presentation of the XML fragments in a stylesheet-like manner. Displaying the whole semantic representation inside Emacs will be also feasible (exactly as the MML Query generated abstracts), and it can be done with very similar methods as the HTML presentation.

3.4 Using the XML Technology for Indexing and Querying

The large-scale adoption of XML in the software industry has another benefit for formal mathematics: it brings new tools for storage and indexing of tree-like structures which typically represent mathematical proofs, formulas and terms. Storing such structures directly e.g. in relational databases is often cumbersome, relying e.g. on full-text indexing of formulas represented as strings, or explicit Parent/Children tables. Several native XML databases centered around tree-like structures are being developed today, e.g. X-Hive¹⁹, Berkeley DB XML²⁰ or eXist²¹. Berkeley DB XML and eXist are probably the most mature freely available systems.

For experimenting with Mizar XML, eXist was chosen. It provides a wider functionality, mainly a working implementation of structural auto-indexing of XML [Mei02] which makes XPath queries efficient without the necessity of low-level manual work on indexing. A very experimental eXist server accepting XQuery queries has been set up²² for the Mizar XML data. Due to limited resources, only the internal Mizar database (with added absolute MML addresses) is now available there. This basically means that all proofs are omitted. The auto-indexing in eXist is based on a numbering scheme which assigns unique identifiers to all nodes in an XML tree. This has to be done in a regular way, so that Ancestor/Descendant and other XPath axes between two nodes (expressed as numbers) were quickly computable. For that purpose the XML tree needs to be “regularized”, i.e. some identifiers are spent on nonexistent nodes. On large or irregular XML documents this can cause that eXist runs out of node identifiers. This limitation made direct eXist storage of practically all XML files with Mizar theorems impossible. The Mizar internal database XML files were therefore split before inserting into eXist, and a separate document was created for each Mizar

¹⁹ <http://www.x-hive.com/>

²⁰ <http://www.sleepycat.com/products/xml.shtml>

²¹ <http://exist.sourceforge.net/>

²² <http://lipa.ms.mff.cuni.cz/~urban/existdemo.html>

resource (theorem, scheme, constructor, etc.). About 90000 smaller XML documents were thus created and loaded into eXist quite successfully: only 332 of these documents were still too irregular for the current eXist auto-indexing. The loading (and indexing which is done on the fly) takes about 30 minutes on an Intel Pentium 4 3.4GHz computer. The size of the resulting database files is about 0.5 GB.

Preliminary experiments show that the Mizar database is still quite large for the eXist server. Particularly, even with the XPath auto-indexing, the path joins done for frequent elements can require portions of memory which are not available on our server. Queries like `//Func` (yields 731154 elements) or `//Typ` (yields 281998 elements) are very fast, and even queries like `//Func/Func` (yielding 327007 elements) and `//Typ//Func` (308399 elements) are fast (below 1 second) and executable within the current memory limit, but e.g. `//Func//Func` is already aborted because of the memory limit. While part of these problems can be solved by having a dedicated server with several GB of memory, generally the eXist system would also have to be augmented to trade some of its speed for lower memory usage in very large XML collections.

Native storage and auto-indexing of XML is a very young field, and while it may be already fully usable for smaller mathematical libraries than MML, custom solutions like MML Query will probably still be necessary for several following years for MML. The functionality provided by languages like XQuery is quite different from the current MML Query language, which has much lower support for tree-like queries, but adds qualifiers like *atmost* and *exactly*. The combination of both should be useful, and building custom search engines like MML Query on top of a native XML database should make a lot of the low-level work on such custom system unnecessary. On the other hand, for very specialized mathematical operations like matching and unification, the ATP-based indexing methods [RSV01] like discrimination trees (used now in the MoMM system) will probably remain superior to the more general XML indexing schemes.

3.5 Data-Mining Queries

Here we give examples of simple XML processing pieces of code that can be used to extract input for various AI tools from MML. First the XSLT and then the XQuery version are shown. XSLT and XQuery will be usable for most of these task, and obviously they can be complemented or embedded into other programming languages for more complex tasks.

a) Get sequences of top-level proof steps for various natural-language-like (e.g. Markov model) learnings:

```
tpl [Proof] { <Proof { for-each [*] { name(); } } }

for $i in //Proof
return <Proof> { for $j in $i/* return name($j) } </Proof>
```

This will yield sequences of rule names like:

```
<Proof>Let Let Assume Consider Take IterEquality Conclusion</Proof>
```

b) For each theorem get all theorems and definitions used in its proof, (should be executed on the richer XML with absolute MML addresses):

```
tpl [JustifiedTheorem] { <Refs
  { for-each [./Ref[@kind]] { '@aid'; ":"; '@kind'; '@absnr'; }}}
for $i in //JustifiedTheorem return
  <Refs>
  { for $j in $i//Ref[@kind] return
    concat(string($j/@aid),":",string($j/@kind),string($j/@absnr))}
  </Refs>
```

This will yield sequences of references like:

```
<Refs>CARD_1:T64 ORDINAL1:T19 ORDINAL1:T23 ORDINAL1:T26</Refs>
```

c) For each theorem get all constructors in it (should be executed on the richer XML with absolute MML addresses):

```
tpl [JustifiedTheorem/Proposition] { <Constrs
  { for-each [./((Pred|Func|Typ|Adjective))]
    { '@aid'; ":"; '@kind'; '@absnr'; }}}
for $i in //JustifiedTheorem/Proposition return
  <Constrs>
  { for $j in $i//((Pred|Func|Typ|Adjective) return
    concat(string($j/@aid),":",string($j/@kind),string($j/@absnr))}
  </Constrs>
```

This will yield sequences of constructors like:

```
<Constrs>CARD_1:V1 HIDDEN:M1 HIDDEN:R2 ORDINAL2:K5</Constrs>
```

The previous two queries put together are actually the input to the Mizar Proof Advisor training (now a Bayesian classifier implemented by the SNoW [CCRR99] system).

4 Conclusions and Future Work

We believe that the switch to XML representation in Mizar was carried out at about the right time. Though still advancing very fast, the XML and derived technologies like XSLT have matured, and have been implemented in many mainstream applications like internet browsers. Standard XML parsers exist for a number of programming languages, implementations of XPath and XQuery standards are becoming quite common, and both relational and native XML

storage and indexing models are being developed quite actively. We have described above several initial Mizar-related applications taking advantage of this new technology. For many simple applications, XSLT processing of Mizar data will be sufficient, and writing a couple of lines in a high-level declarative language like XSLT_{XT} should be much easier than building on the Pascal implementation of Mizar. Quite an important difference between this work and earlier Mizar-exporting tools is that the XML format is now also used internally by Mizar. It means that the XML format will follow the changes introduced during the fast development of Mizar. Third-party developers will therefore no longer be stuck with outdated exporting tools that no longer work on the newest MML, they can easily detect recent XML changes with tools like `xmldiff`, and keep their external tools as much up-to-date as they wish. The user-visible increase in the verification time is not negligible, however it is neither dramatic, especially in the view of the drop that occurred in the past years thanks to the general growth of computing power.

The largest future extension of the work presented here should be the above mentioned inclusion of the Mizar presentation layer. This should allow simple reimplementations of presentation tools like the Journal of Formalized Mathematics, and improve the current content-derived presentation methods used for direct browsing or in MML Query. Another possible extension is detailed reporting about the smallest verification steps done by Mizar, e.g., in a way similar to the Otter [McC94] proof objects. Some preliminary work has been done in this direction, however this practically amounts to a large-scale reimplementations of the Mizar proof-checking module. A cheaper (though less elegant) alternative how to do this can be e.g. using external automated theorem provers like Otter to re-prove the Mizar verification steps, and get these detailed proof objects from them. This would have the additional cross-verification benefit, and is planned as a part of the MPTP project.

The Mizar XML format is now very fresh, and is likely to go through some more changes, particularly in order to unify the remaining redundancies and to make it more compact. We hope that this article will also stimulate feedback from potential users of the Mizar library, which is a unique source for a number of data-mining, automated reasoning, presentational, and information retrieval experiments.

Acknowledgments

The idea of using XML for a number of Mizar-related tasks has been thoroughly discussed within the Mizar team, naming at least Andrzej Trybulec, Czesław Byliński and Grzegorz Bancerek. The final step that triggered the work described here was the implementation of a Tiny-XML Pascal parser for Mizar by Czesław Byliński. The recent versions of the HTML presentation contain many improvements suggested by Jiří Vyskočil. This work was partially supported by the CALCULEMUS Research Training Network (HPRN-CT-2000-00102) and Charles University research grants (205-03/2060985, 205-10/203336).

References

- [BR03] Grzegorz Bancerek and Piotr Rudnicki. Information retrieval in MML. In *MKM*, volume 2594 of *Lecture Notes in Computer Science*, pages 119–132. Springer, 2003.
- [BU04] Grzegorz Bancerek and Josef Urban. Integrated semantic browsing of the Mizar Mathematical Library for authoring Mizar articles. In *MKM*, volume 3119 of *Lecture Notes in Computer Science*, pages 44–57. Springer, 2004.
- [CCRR99] A. J. Carlson, C. M. Cumby, J. L. Rosen, and D. Roth. Snow user’s guide. Technical Report UIUC-DCS-R-99-210, UIUC, 1999.
- [Dah98] Ingo Dahn. Interpretation of a Mizar-like logic in first-order logic. In *FTP (LNCS Selection)*, pages 137–151, 1998.
- [Jas34] S. Jaskowski. On the rules of suppositions. *Studia Logica*, 1, 1934.
- [McC94] W. W. McCune. Otter 3.0 reference manual and guide. Technical Report ANL-94/6, Argonne National Laboratory, Argonne, Illinois, 1994.
- [Mei02] Wolfgang Meier. eXist: An open source native XML database. In *Web, Web-Services, and Database Systems*, volume 2593 of *Lecture Notes in Computer Science*, pages 169–183. Springer, 2002.
- [MR04] Roman Matuszewski and Piotr Rudnicki. Mizar: the first 30 years. In Grzegorz Bancerek, editor, *MKM Workshop on 30 Years of Mizar*, 2004.
- [Pel99] F. J. Pelletier. A brief history of natural deduction. *History and Philosophy of Logic*, 20:1 – 31, 1999.
- [RSV01] I.V. Ramakrishnan, R. Sekar, and A. Voronkov. *Handbook of Automated Reasoning*, chapter Term Indexing, pages 1853–1964. Elsevier and MIT Press, 2001.
- [RT99] Piotr Rudnicki and Andrzej Trybulec. On equivalents of well-foundedness. *J. Autom. Reasoning*, 23(3-4):197–234, 1999.
- [Rud92] P. Rudnicki. An overview of the Mizar project. In *1992 Workshop on Types for Proofs and Programs*, pages 311–332. Chalmers University of Technology, Bastad, 1992.
- [Urb04] Josef Urban. MPTP - motivation, implementation, first experiments. *Journal of Automated Reasoning*, 33(3-4):319–339, 2004.
- [Urb05a] Josef Urban. MizarMode - an integrated proof assistance tool for the Mizar way of formalizing mathematics. *Journal of Applied Logic*, 2005. forthcoming, available online at <http://ktiml.mff.cuni.cz/~urban/mizmode.ps>.
- [Urb05b] Josef Urban. MoMM - fast interreduction and retrieval in large libraries of formalized mathematics. *International Journal on Artificial Intelligence Tools*, 2005. forthcoming, available online at <http://ktiml.mff.cuni.cz/~urban/MoMM/momm.ps>.